

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. ШУХОВА» (БГТУ им. В.Г. Шухова)
Кафедра программного обеспечения
вычислительной техники и автоматизированных
систем

Лабораторная работа №1
по дисциплине: Архитектура вычислительных систем
тема: «**Разработка программ на ассемблере.**
Работа с отладчиком x32dbg, пакетом nasm32»

Выполнил: студент группы ВТ-231
Масленников Д. А.
Проверили:
Осипов О.В.
Бондаренко Т.В.

Белгород 2025

Цель работы: получить навыки создания простейших ассемблерных программ с использованием пакета `masm32` и научиться пользоваться отладчиком `x32dbg`.

Примечание: на лекции было сказано о том, что при выполнении лабораторной работы можно использовать операционную систему линукс. Поэтому, так как я привык работать именно в ней, лабораторная будет выполнена с использованием возможностей этой системы — в частности компилятор `masm`, а так же отладчик `gdb`

Задание по варианту №14:

Исходный код на MASM32:

```
.DATA
String1 DB 13, 10, "____", 0
ds DW 5
result DD ?
p DQ 17.5
ten DT 183333.5
.CODE
START:
    XOR ECX, ECX
    MOV CX, ds
    ADD CX, 5
    MOV EAX, ECX
    DIV 2
    MOV result, EAX
END START
```

Исходный код программы на NASM для Linux:

```
section .data                ; секция инициализированных данных
    String1 db 13, 10, "____", 0 ; строка с символами возврата каретки(13) и перевода строки(10)
    data_size dw 5             ; переменная размером 2 байта со значением 5 (аналог ds из MASM32)
    align 4                   ; выравнивание следующей метки на границу 4 байта
    result dd 0                ; переменная размером 4 байта для хранения результата (инициализирована 0)
    p dq 17.5                 ; переменная размером 8 байт со значением 17.5
    ten dt 183333.5           ; переменная размером 10 байт со значением 183333.5

section .text                 ; секция кода
    global _start             ; объявление точки входа для линкера

_start:                       ; метка начала программы
    xor ecx, ecx               ; обнуление регистра ecx
    movzx ecx, word [data_size] ; загрузка значения из data_size в регистр ecx с расширением нулём
    add ecx, 5                 ; увеличение значения в регистре ecx на 5
    mov eax, ecx               ; копирование значения из ecx в eax (для операции деления)
    mov edx, 0                 ; обнуление регистра edx (обязательно для операции деления)
    mov ebx, 2                 ; загрузка значения 2 в регистр ebx (делитель)
    div ebx                    ; деление eax на ebx, результат в eax, остаток в edx
    mov [result], eax          ; сохранение результата деления в переменную result

    mov eax, 4                 ; системный вызов write (4)
    mov ebx, 1                 ; файловый дескриптор stdout (1)
    mov ecx, String1           ; указатель на строку
    mov edx, 6                 ; длина строки (6 байт: CR, LF + 4 символа)
    int 0x80                   ; вызов прерывания

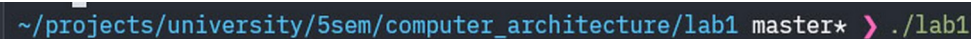
    mov eax, 1                 ; номер системного вызова exit
    xor ebx, ebx               ; код возврата 0 (успешное завершение)
    int 0x80                   ; вызов прерывания для завершения программы
```

Компиляция и запуск:

Для компиляции и запуска программы на Linux с использованием NASM и LD используются следующие команды bash:

```
nasm -f elf32 -g -F dwarf lab1.asm -o lab1.o
ld -m elf_i386 lab1.o -o lab1
./lab1
```

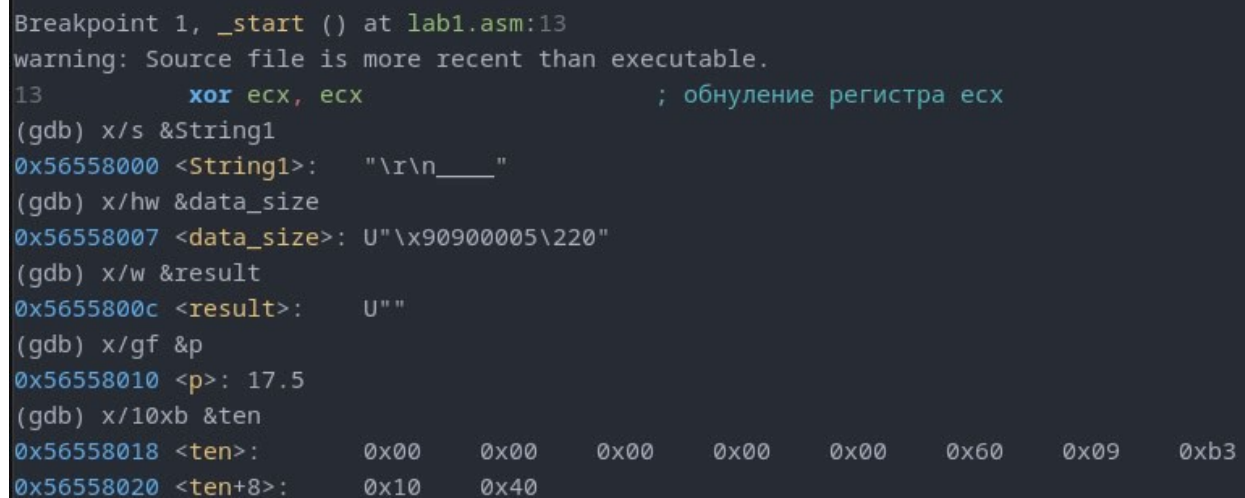
Результат выполнения



```
~/projects/university/5sem/computer_architecture/lab1 master* > ./lab1
```

Код выводит строку согласно одному из подпунктов пункта 4

Анализ сегмента данных с помощью GDB:



```
Breakpoint 1, _start () at lab1.asm:13
warning: Source file is more recent than executable.
13      xor ecx, ecx                                ; обнуление регистра ecx
(gdb) x/s &String1
0x56558000 <String1>:  "\r\n____"
(gdb) x/hw &data_size
0x56558007 <data_size>:  U"\x90900005\220"
(gdb) x/w &result
0x5655800c <result>:  U""
(gdb) x/gf &p
0x56558010 <p>:  17.5
(gdb) x/10xb &ten
0x56558018 <ten>:  0x00  0x00  0x00  0x00  0x00  0x60  0x09  0xb3
0x56558020 <ten+8>:  0x10  0x40
```

Описание:

Мы переходим на `_start` программы, где у нас уже записались все данные. После этого мы просматриваем все переменные, получая информацию о сегменте данных и значениях. При получении этой информации стоит не забывать использовать соответствующий заданному типу атрибут

Результаты анализа:

1. Строка String1

- Адрес: 0x56558000
- Содержимое: 0x0d 0x0a 0x5f 0x5f 0x5f 0x5f 0x00
- Размер: 7 байт
- ASCII: "\r\n____\0"

2. Переменная data_size

- Адрес: 0x56558007
- Значение: 5 (0x0005)
- Размер: 2 байта

3. Переменная result (до вычислений)

- Адрес: 0x5655800c
- Значение: 0 (0x00000000)
- Размер: 4 байта

4. Переменная p

- Адрес: 0x56558010
- Значение: 17.5 (в формате double)
- Байтовое представление: 0x00 0x00 0x00 0x00 0x00 0x00 0x31 0x40
- Размер: 8 байт

5. Переменная ten

- Адрес: 0x56558018
- Значение: 183333.5 (в формате extended precision)
- Байтовое представление: 0x00 0x00 0x00 0x00 0x00 0x6e 0xb2 0x40 0x00 0x40
- Размер: 10 байт

Дамп памяти:

```
(gdb) x /34xb 0x0804a000
0x0804a000: 0x0d 0x0a 0x5f 0x5f 0x5f 0x5f 0x00 0x05
0x0804a008: 0x00 0x90 0x90 0x90 0x00 0x00 0x00 0x00
0x0804a010: 0x00 0x00 0x00 0x00 0x00 0x80 0x31 0x40
0x0804a018: 0x00 0x00 0x00 0x00 0x00 0x60 0x09 0xb3
0x0804a020: 0x10 0x40
```

Адрес	Шестнадцатеричное	Интерпретация
0x0804a000	0x0d 0x0a 0x5f 0x5f 0x5f 0x5f 0x00 0x05	String1: '\r\n____\0' + data_size = 5
0x0804a008	0x00 0x90 0x90 0x90 0x00 0x00 0x00 0x00	Выравнивание (пор-байты) + нулевые байты
0x0804a010	0x00 0x00 0x00 0x00 0x00 0x80 0x31 0x40	result = 0 (4 байта), p = 17.5 (начало)
0x0804a018	0x00 0x00 0x00 0x00 0x00 0x60 0x09 0xb3	p = 17.5 (продолжение), ten = 183333.5 (начало)
0x0804a020	0x10 0x40	ten = 183333.5 (окончание)

Пошаговая трассировка программы:

Запускаем программу в GDB и пошагово выполняем инструкции, наблюдая за изменениями регистров и памяти.

```
Breakpoint 1, _start () at lab1.asm:13
warning: Source file is more recent than executable.
13      xor ecx, ecx                ; обнуление регистра ecx
(gdb) x/w &result
0x5655800c <result>:  0
(gdb) stepi
14      movzx ecx, word [data_size] ; загрузка значения из data_size в регистр ecx с расширением нулём
(gdb) info registers
eax                0x38                56
ecx                0x0                 0
edx                0xf7fcc080          -134430592
ebx                0xf7ffcfec          -134230036
esp                0xffffccd0          0xffffccd0
ebp                0x0                 0x0
esi                0xffffccdc          -13092
edi                0x56556000          1448435712
eip                0x56556002          0x56556002 <_start+2>
eflags            0x246                [ PF ZF IF ]
cs                0x23                 35
ss                0x2b                 43
ds                0x2b                 43
es                0x2b                 43
fs                0x0                 0
gs                0x63                 99
```

```
(gdb) stepi
16      mov eax, ecx                ; копирование значения из ecx в eax (для опер
(gdb) info registers
eax                0x38                56
ecx                0xa                 10
edx                0xf7fcc080          -134430592
ebx                0xf7ffcfec          -134230036
esp                0xffffccd0          0xffffccd0
ebp                0x0                 0x0
esi                0xffffccdc          -13092
edi                0x56556000          1448435712
eip                0x5655600c          0x5655600c <_start+12>
eflags            0x206                [ PF IF ]
cs                0x23                 35
ss                0x2b                 43
ds                0x2b                 43
es                0x2b                 43
fs                0x0                 0
gs                0x63                 99
```

```
(gdb) stepi
18      mov ebx, 2                ; загрузка значения 2 в регистр ebx (делитель)
(gdb) info registers
eax                0xa                10
ecx                0xa                10
edx                0x0                0
ebx                0xf7ffcfcfc        -134230036
esp                0xffffccd0         0xffffccd0
ebp                0x0                0x0
esi                0xffffccdc        -13092
edi                0x56556000         1448435712
eip                0x56556013         0x56556013 <_start+19>
eflags            0x206                [ PF IF ]
cs                 0x23                35
ss                 0x2b                43
ds                 0x2b                43
es                 0x2b                43
fs                 0x0                0
gs                 0x63                99
```

```
(gdb) stepi
19      div ebx                ; деление eax на ebx, результат в eax, остаток в edx
(gdb) info registers
eax          0xa             10
ecx          0xa             10
edx          0x0             0
ebx          0x2             2
esp          0xffffccd0      0xffffccd0
ebp          0x0             0x0
esi          0xffffccdc      -13092
edi          0x56556000      1448435712
eip          0x56556018      0x56556018 <_start+24>
eflags       0x206           [ PF IF ]
cs           0x23            35
ss           0x2b            43
ds           0x2b            43
es           0x2b            43
fs           0x0             0
gs           0x63            99
```



```

(gdb) stepi
20      mov [result], eax          ; сохранение результата деления в переменную result
(gdb) info registers
eax      0x5                      5
ecx      0xa                      10
edx      0x0                      0
ebx      0x2                      2
esp      0xffffccd0              0xffffccd0
ebp      0x0                      0x0
esi      0xffffccdc              -13092
edi      0x56556000              1448435712
eip      0x5655601a              0x5655601a <_start+26>
eflags   0x212                   [ AF IF ]
cs       0x23                     35
ss       0x2b                     43
ds       0x2b                     43
es       0x2b                     43
fs       0x0                      0
gs       0x63                     99
(gdb) stepi
22      mov eax, 1                ; номер системного вызова exit
(gdb) x/w &result
0x5655800c <result>:    5

```

Программа завершается с кодом 0.

Выводы:

В ходе лабораторной работы была написана программа на ассемблере NASM для платформы Linux, которая выполняет арифметические операции (сложение и деление) и сохраняет результат в переменную. Программа была успешно скомпилирована и запущена. С помощью отладчика GDB был проведен анализ сегмента данных и пошаговая трассировка программы, в ходе которой было подтверждено корректное выполнение всех инструкций. Получены навыки работы с отладчиком и анализа памяти.